# 4. Hafta

# Kaynak Yönetimi ve Akıllı İşaretçiler

**◎ Öğrenme Hedefleri**

Bu bölümü tamamladığınızda:

- Bir sınıf ham işaretçi/kaynak tutuyorsa derleyicinin ürettiği varsayılan (sığ) kopyanın neden tehlikeli olduğunu (çifte `delete`, paylaşılan durum) açıklayabileceksiniz;
- Kaynak yöneten bir sınıf için doğru bir kopya kurucu ve kopya atama operatörü (derin kopya) yazabileceksiniz;
- Rule of 3'ü uygulayabilecek, `=default`/`=delete` sözdizimini kullanabilecek ve taşıma (move) semantiğinin ne işe yaradığını en azından kavramsal düzeyde açıklayabileceksiniz;
- `unique_ptr` ve `shared_ptr`'ı, `make_unique`/ `make_shared` ile birlikte, elle `new`/`delete` yönetimi yerine kullanabileceksiniz;
- Düğümlerini akıllı işaretçilerle yöneten basit bir bağlı liste (`IntBagliListe`) tasarlayıp uygulayabileceksiniz.

## 4.1 Kopyalama Sorunu: Sığ Kopya Neden Tehlikeli?

**1 • Ön-öğretim 📢 Neden Önemli?**

Hafta 2'de `araclar::IntTampon`'u kurdunuz — kurucusu `new[]` ile bellek ayırıyor, yıkıcısı `delete[]` ile bırakıyordu; ama onu hiç kopyalamamıştınız. Şimdi masum görünen tek bir satır — `IntTampon b = a;` — yazın: derleyici hiç uyarmadan iki nesneyi aynı belleğe bağlar, program bittiğinde aynı blok iki kez silinmeye çalışılır. Kopyalama, üzerinde hiç durmadığınız bu sessiz tuzağı taşır.

**1 • Ön-öğretim 🕘 Ön Bilgi**

Bu üniteye girmeden önce Hafta 2'nin `araclar::IntTampon`'unu (ham `int* veri`, `int boyut`; `new[]`/`delete[]`) ve Hafta 3'ün kurucu/yıkıcı biçimsel terimlerini (üye başlatma listesi, `this`, sınıf ve kapsülleme) hatırlamanız gerekir. Bu ünite `IntTampon`'a kopya kurucu ve kopya atama operatörünü ekleyerek onu tamamlar.

Geçen hafta `Kitap` sınıfıyla kurucu ve yıkıcının biçimsel kurallarını — üye başlatma listesi, `this`, kapsam sonunda otomatik çalışan yıkıcı — gördünüz. O ünitenin kapanışındaki kısa değini bilerek bir soruyu açık bırakmıştı: bir nesne bellek gibi bir *kaynağa* sahipse, bu kaynağı yalnızca doğru kurup doğru yıkmak yeterli midir, yoksa başka bir şey daha mı gerekir? Bu haftanın çıkış noktası tam olarak bu sorudur — ve cevabını Hafta 2'de tanıdığınız `araclar::IntTampon` üzerinden bulacaksınız.

`IntTampon`'u hatırlayın: öbekte bir tam sayı dizisi tutan, ham bir işaretçi ve bir boyut alanından oluşan küçük bir sarmalayıcı.

```
8     int* veri;
9     int boyut;
```

Kurucusu `new[]` ile diziyi ayırır, yıkıcısı `delete[]` ile bırakır — bu iki özel üye fonksiyonun adlarını (kurucu, yıkıcı) ve biçimsel yazım kurallarını Hafta 3'te resmî olarak öğrendiniz, gövdeleri de Hafta 2'deki hâliyle aynıdır. Ama iki hafta boyunca hiç sorulmayan bir soru kaldı: `IntTampon` tipinden bir nesneyi *kopyaladığınızda* tam olarak ne olur?

### 4.1.1 Derleyicinin sessizce ürettiği kopya

Şu satırları bir arada düşünün:

```
1 araclar::IntTampon a(3);
2 araclar::IntTampon b = a; // bu satir derlenir -- ama ne yapar?
```

İkinci satırı hiç ekleyecek bir kod yazmadınız — ne `IntTampon` sınıfına, ne başka bir yere. Yine de bu satır sorunsuz derlenir. Nedeni basit bir C++ kuralıdır: bir sınıf kendi kopya kurucusunu ve kopya atama operatörünü *tanımlamazsa*, derleyici bunları sizin için kendisi üretir; siz hiçbir şey yazmasanız bile `IntTampon b = a;` gibi bir satır her zaman derlenir, çünkü çağıracağı bir kopya kurucu (üretilmiş de olsa) her zaman vardır.

**Tanım: Varsayılan (derleyici üretimi) kopya kurucu ve kopya atama operatörü**

Bir sınıf kopya kurucusunu ve kopya atama operatörünü kendisi tanımlamazsa, derleyici her ikisini de otomatik üretir. Bu üretilmiş sürümler *üye üye* (*memberwise*) çalışır: kaynak nesnenin her alanı, hedef nesnenin aynı adlı alanına tek tek kopyalanır — alan bir `int` ise değeri, alan bir işaretçi ise *işaretçinin kendisi* (yani tuttuğu adres) kopyalanır.

Bu son cümledeki ayrıntıyı — "işaretçinin kendisi kopyalanır, işaret ettiği veri değil" — gözden kaçırmayın, çünkü tehlike tam orada saklıdır. `IntTampon`'un iki alanını tek tek düşünün: `boyut` bir `int`'tir, üye üye kopyalama onun için sorunsuzdur — `b.boyut`, `a.boyut` ile aynı sayısal değeri alır, ikisi artık birbirinden tamamen bağımsız iki kopyadır. Ama `veri` bir `int*`'tir; üye üye kopyalama burada da aynı kuralı uygular ve yalnızca adresin *kendisini* kopyalar. Sonuç olarak `b.veri` ile `a.veri` aynı sayısal değeri — öbekteki *aynı* bellek adresini — taşır. Yeni bir dizi hiç ayrılmaz; `b` kendi verisine sahip olmaz, yalnızca `a`'nın verisine *bakan* ikinci bir işaretçi kazanır. İşte bu duruma — üyelerin, özellikle işaretçilerin, işaret ettikleri veriyi paylaştıracak biçimde yalnızca adres düzeyinde kopyalanmasına — **sığ kopya** denir.

**sığ kopya**: üyelerin (özellikle işaretçilerin) yalnızca değerini/adresini kopyalayan, işaret edilen veriyi paylaştıran kopya (*shallow copy*).

**Analoji**

Sığ kopyayı aynı eve gönderilen iki zarfa benzetebilirsiniz. İki zarfın üzerinde de — iki nesnenin de — yazan adres birebir aynıdır; ikisi de aynı eve (aynı bellek bloğuna) açılır. Zarfları elinde tutan iki kişiden biri "bu evi yık" derse, öteki zarfın sahibi de artık evsiz kalır — onun elindeki adres artık geçersiz bir eve işaret eder. İkisi de aynı anda "evi yık" derse, aynı ev iki kez yıkılmaya çalışılır; gerçek hayatta bu saçma bir durumdur, ama bellekte tam karşılığı vardır: aynı adresin iki kez `delete[]` edilmesi.
Şimdi bu benzetmeyi doğrudan koda eşleyin. Zarf, bir `IntTampon` nesnesidir; zarfın üzerindeki adres, `veri` işaretçisinin tuttuğu değerdir; ev ise öbekte `new[]` ile ayrılmış tek bir bellek bloğudur. Sığ kopyada iki zarf (iki nesne) aynı evin adresini taşır — ev bir tanedir, ona "bakan" zarf ikidir.

### 4.1.2 Sonuç: paylaşılan durum ve çifte `delete`

Bu paylaşımın pratikte iki somut sonucu vardır ve ikisini de adım adım izleyin. **Birincisi**, `a` ve `b` artık aynı diziyi gösterdiği için, `a.veri[0] = 99;` gibi bir satır `a` üzerinden yazılmış olsa da, `b.veri[0]`'ı okuduğunuzda da `99` görürsünüz — iki bağımsız nesne beklerken, elinizde tek bir veriyi paylaşan iki "görünüm" vardır; biri değiştirirse öteki de değişmiş gibi görünür. Bu, çoğu zaman istenmeyen, sessiz bir *paylaşılan durum* sorunudur. **İkincisi**, ve daha ciddisi, kapsam sonunda gelir: `a` ile `b` kapsam dışına çıktığında, ikisinin de yıkıcısı otomatik çalışır ve ikisi de kendi `veri`'sini `delete[]` eder — ama `veri` ikisinde de aynı adrestir. Aynı bellek bloğu böylece *iki kez* `delete[]` edilmiş olur; bu, C++'ta tanımsız davranıştır (bellek yöneticisinin iç defterini bozabilir, programı çökertebilir ya da sessizce yanlış çalışmaya devam ettirebilir).

Bu riski soyut bırakmayın; kaynak kodunda gerçek bir yorum bloğu olarak duran şu açıklamayı satır satır okuyun:

```
27    // Not: IntTampon icin kullanici tanimli kopya kurucu/atama olmasaydi,
28    // derleyici varsayilan (sig/uye-uye) kopyayi uretirdi. O durumda
29    // `araclar::IntTampon x = a;` ifadesinde x.veri ve a.veri AYNI heap
30    // adresini gosterirdi (yalnizca isaretci/adres kopyalanir, isaret
31    // edilen veri kopyalanmaz). Ikisi de kapsam disina ciktiginda ayni
32    // bellek blogu iki kez `delete[]` edilirdi (cifte delete, tanimsiz
33    // davranis). Bu kesit yalnizca desen/aciklama icindir; gercek bir
34    // tanimsiz davranis burada TETIKLENMEZ.
```

Dikkat edin: bu kesit yalnızca bir *açıklama*, gerçek bir çalışan kod değildir — Hafta 2'den tanıdığınız kuralın burada da geçerli olduğunu unutmayın: ders notundaki hiçbir kesit gerçek bir tanımsız davranışı tetiklemez, derlenebilir ve güvenle çalıştırılabilir kalır. Kesit yalnızca deseni — kullanıcı tanımlı bir kopya kurucu olmasaydı ne olacağını — açıklayıcı yorum olarak gösterir.

### 4.1.3 Çözüm: kendi kopyanızı elle yazmak

Bu tehlikeyi ortadan kaldırmanın yolu, derleyicinin ürettiği varsayılan davranışa güvenmek yerine, `IntTampon`'a iki özel üye fonksiyonu kendiniz eklemektir. Bildirimleri sınıfın içine şöyle eklersiniz:

```
16    IntTampon(const IntTampon& diger);
17    IntTampon& operator=(const IntTampon& diger);
```

**kopya kurucu**: var olan bir nesneden yeni bir nesne kurarken çağrılan özel üye fonksiyon (*copy constructor*); imzası her zaman `Sinif(const Sinif& diger)` biçimindedir.

İlk bildirim — `IntTampon`'un aynı tipten bir nesneyi (`const` referansla, değiştirmeden) parametre alan kurucusu — bir **kopya kurucu**dur: tam olarak `IntTampon b = a;` gibi bir satırda — yeni bir `b` nesnesi *var olan* bir `a`'dan kurulurken — devreye giren fonksiyondur. İkinci bildirim ise bir *kopya atama operatörü*dür (adını birazdan göreceksiniz): `c = a;` gibi — `c` zaten var olan bir nesneyken üzerine `a`'nın içeriği yazılırken — devreye giren fonksiyondur. İkisi arasındaki fark inceliklidir ama kritiktir: kopya kurucu *yeni* bir nesne doğururken çalışır, kopya atama operatörü ise *zaten var olan* bir nesnenin üzerine yazarken çalışır — bu farkın atama operatöründe ek bir sorumluluk doğurduğunu birazdan göreceksiniz.

Kopya kurucunun gövdesini adım adım okuyun:

```
23 IntTampon::IntTampon(const IntTampon& diger) {
24    boyut = diger.boyut;
25    veri = new int[boyut];
26    for (int i = 0; i < boyut; ++i) {
27       veri[i] = diger.veri[i];
28    }
29    std::cout << "tampon kopyalandi (derin kopya, boyut=" << boyut << ")\n";
30 }
```

Bu gövde, sığ kopyanın yaptığından kökten farklı bir şey yapar: önce `diger.boyut`'u kendi `boyut`'una kopyalar (bu kısım hâlâ üye üyedir, `int` için sorun değildir), ama `veri` için adresi kopyalamak yerine `new int[boyut]` ile kendi *yeni* bir dizi ayırır, ardından bir döngüyle `diger.veri`'nin elemanlarını tek tek kendi `veri`'sine kopyalar. Sonuçta iki nesne artık aynı adresi değil, aynı *içeriğe* sahip ama bellekte tamamen ayrı iki dizi tutar. Bu işleme — işaret edilen veriyi yeniden ayırıp elemanlarıyla kopyalayarak bağımsız iki kaynak üretmeye — **derin kopya** denir; artık `a.veri[0] = 99;` yazsanız bile `b.veri[0]` hiç etkilenmez, çünkü ikisi farklı bellek bloklarını gösterir.

**derin kopya**: işaret edilen veriyi de yeniden ayırıp kopyalayan, bağımsız iki kaynak üreten kopya (*deep copy*).

**Analoji**

Derin kopyayı, bir evin birebir bir kopyasını — aynı plan, aynı mobilyalar — *yeni bir arsada* baştan inşa etmeye benzetebilirsiniz. İnşaat bittiğinde artık iki bağımsız ev vardır; ikisi de aynı görünse bile, biri yıkılsa öteki hiç etkilenmez, çünkü aralarında ortak hiçbir temel yoktur. `IntTampon`'un kopya kurucusu tam olarak bunu yapar: `a`'nın verisine bakan ikinci bir zarf oluşturmak yerine, `a`'nın verisinin tıpatıp aynı içerikli *yeni* bir kopyasını kendi öbek alanında inşa eder.

Kopya atama operatörünün gövdesi de benzer bir derin kopya yapar, ama bir adım daha barındırır:

```
34 IntTampon& IntTampon::operator=(const IntTampon& diger) {
35    if (this == &diger) {
36       return *this; // oz-atama kontrolu
37    }
38    delete[] veri; // eski kaynak birakilir
39    boyut = diger.boyut;
40    veri = new int[boyut];
41    for (int i = 0; i < boyut; ++i) {
42       veri[i] = diger.veri[i];
43    }
44    std::cout << "tampon atandi (derin kopya, boyut=" << boyut << ")\n";
45    return *this;
46 }
```

**kopya atama operatörü**: var olan bir nesneye başka bir nesnenin içeriğini atarken çağrılan özel üye fonksiyon (*copy assignment operator*); imzası her zaman `Sinif& operator=(const Sinif& diger)` biçimindedir.

Bu gövde bir **kopya atama operatörü**dür ve onu kopya kurucudan ayıran ilk satıra dikkat edin: `if (this == &diger)`. Bu kontrole *öz-atama* (*self-assignment*) kontrolü denir ve `c = c;` gibi (görünüşte anlamsız ama geçerli) bir atamada devreye girer. Bu kontrolün neden gerekli olduğunu adım adım düşünün: eğer bu satır olmasaydı, fonksiyon doğrudan `delete[] veri;` çalıştırırdı — ama `this` ile `&diger` aynı nesneyi gösterdiğinde, `veri` ile `diger.veri` da aynı adrestir; bu adresi `delete[]` etmek, birazdan *okumaya çalışacağınız* `diger.veri`'yi de geçersiz kılar — derin kopya yapmak için artık var olmayan bir kaynaktan okumaya çalışırsınız. `this == &diger` doğruysa fonksiyon hiçbir şey yapmadan doğrudan `*this`'i döndürür; nesnenin kendi kendine atanması zaten hiçbir değişiklik gerektirmez. Kontrolden sonraki adımlar sırayla şöyledir: önce `delete[] veri;` ile *eski* kaynak bırakılır (aksi hâlde eski dizi hiç serbest bırakılmadan üzerine yeni bir `new[]` atanır — bellek sızıntısı), sonra kopya kurucudaki aynı derin kopya adımları — yeni `new[]`, eleman eleman kopyalama — tekrarlanır, son olarak `*this` döndürülür (bu, `c = a = b;` gibi zincirleme atamaları mümkün kılar).

**Ortak Hata / Tuzak**

Öz-atama kontrolünü atlamak, kopya atama operatöründe en sık düşülen tuzaktır — çünkü `c = c;` gibi bir satırı kimse kasıtlı yazmaz, ama dolaylı yoldan (bir dizinin elemanını kendisine atamak, bir işaretçinin gösterdiği nesneyi kendisine atamak gibi) sessizce oluşabilir. Kontrol olmadan yukarıdaki gövde şu sırayla çalışır: `delete[] veri;` çalışır (kaynak bırakılır), ama `diger` de `this` ile aynı nesne olduğu için `diger.veri` de artık geçersiz bir adrestir; hemen ardından `diger.boyut` okunur (bu hâlâ geçerlidir, `int` bırakılmadı) ve döngü `diger.veri[i]`'yi okumaya çalışır — ama bu artık `delete[]` edilmiş, geçersiz bir bellek adresine erişmektir. Tek satırlık bir `if (this == &diger) return *this;` kontrolü bu riski baştan kapatır; bu yüzden her kopya atama operatöründe ilk satır olarak alışkanlık hâline getirmeniz gerekir.

Bu iki fonksiyonu gerçek kullanımda görün:

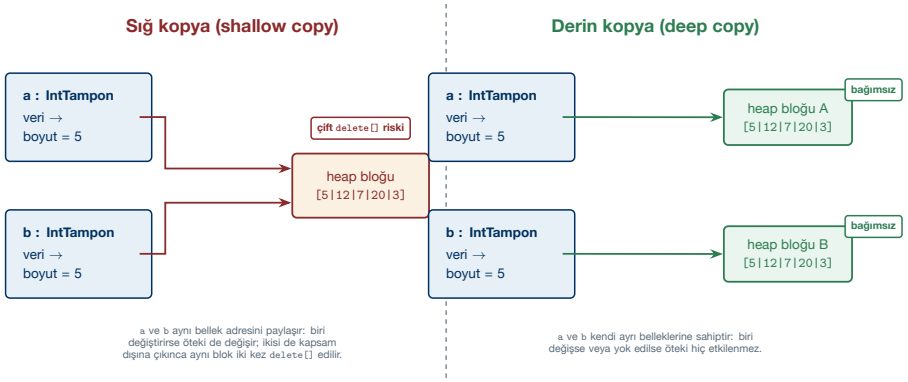
```
8   std::cout << "-- kopya kurucu demo --\n";
9   araclar::IntTampon a(3);
10  for (int i = 0; i < a.boyut; ++i) {
11     a.veri[i] = i + 1; // a = {1,2,3}
12  }
13  araclar::IntTampon b = a; // kopya kurucu cagrilir (derin kopya)
14  a.veri[0] = 99; // yalnizca a degisir
15  std::cout << "a.veri[0]=" << a.veri[0] << " b.veri[0]=" << b.veri[0] << "\
        n";
```

Bu satırları izleyin: `a` üç elemanla kurulur ve doldurulur; `IntTampon b = a;` artık — derleyicinin ürettiği sığ kopya yerine — az önce yazdığınız kopya kurucuyu çağırır, dolayısıyla `b` kendi bağımsız dizisini alır. `a.veri[0] = 99;` yalnızca `a`'yı değiştirir; hemen ardından yazdırılan `b.veri[0]` hâlâ `1`'dir — iki nesne artık gerçekten bağımsızdır. Atama operatörü de aynı bağımsızlığı sağlar:

```
19  std::cout << "-- kopya atama demo --\n";
20  araclar::IntTampon c(3);
21  c = a; // kopya atama operatoru cagrilir (derin kopya)
22  a.veri[1] = 42; // yalnizca a tekrar degisir, c etkilenmez
23  std::cout << "a.veri[1]=" << a.veri[1] << " c.veri[1]=" << c.veri[1] << "\
        n";
```

`c = a;` çağrısı kopya atama operatörünü tetikler; `c` artık `a` ile aynı içeriğe ama ayrı bir kaynağa sahiptir. `a.veri[1] = 42;` yalnızca `a`'yı değiştirir, `c.veri[1]` etkilenmeden 2 kalır.

Şekil 4.1 bu iki kopya türünü yan yana özetliyor: solda sığ kopyada iki nesnenin aynı bellek bloğunu paylaşması ve çift `delete[]` riski, sağda derin kopyada her nesnenin kendi bağımsız bloğuna sahip olması.




Şekil 4.1: Sığ kopya (solda): iki nesne aynı bellek bloğunu paylaşır, çift `delete[]` riski doğar. Derin kopya (sağda): her nesne kendi bağımsız bloğuna sahiptir.

Bu ünitede gördüğünüz fikir kısaca şudur: bir sınıf ham bir işaretçi gibi bir kaynağı elle tutuyorsa, derleyicinin size hediye ettiği varsayılan kopya kurucu ve kopya atama operatörü tam da bu kaynak için yanlış şeyi yapar — adresi kopyalar, veriyi kopyalamaz — ve bu, hem istenmeyen paylaşılan duruma hem de çifte `delete`'e yol açabilir. Çözüm, kopya kurucu ve kopya atama operatörünü kendiniz yazıp her ikisinde de derin kopya yapmaktır; atama operatöründe ayrıca öz-atama kontrolü ve eski kaynağı bırakma adımı gerekir. Gelecek ünitede bunun tek başına yeterli olmadığını — `IntTampon` artık üç özel üye fonksiyonu (yıkıcı, kopya kurucu, kopya atama) elle taşıdığı için bu üçünün birlikte tutarlı kalması gerektiğini — göreceksiniz.

**3 • Katılım ⌨ Şimdi Siz Deneyin**

Değerleri 1, 2, 3 olan üç elemanlı bir `IntTampon` nesnesi `a` olsun. (a) `IntTampon` kendi kopya kurucusunu yazmazsa, `IntTampon b = a;` satırı çalışır mı, çalışırsa ne olur? (b) Doğru bir derin kopya kopya kurucusu yazarsanız, `a.veri[0] = 99;` sonrası `b.veri[0]` ne olur?

---

**Geribildirim.** (a) Kendi kopya kurucunuz yoksa satır *derlenir*, ama derleyici üye üye (sığ) kopya üretir: `b.veri`, `a.veri` ile *aynı* adresi tutar. İki nesne aynı bloğu paylaşır; `a.veri[0] = 99;` dediğinizde `b.veri[0]` de 99 olur ve ikisi kapsam dışına çıkınca aynı blok iki kez `delete[]` edilir. (b) Kendi kopya kurucunuz kendi `new[]`'ini yapıp veriyi eleman eleman kopyalarsa (derin kopya), `b` bağımsız olur: `a.veri[0] = 99;` sonrası `b.veri[0]` hâlâ 1'dir.

**Kendinizi Sınayın 4.1**

Bir sınıf ham bir işaretçi/kaynak tutuyorsa, derleyicinin ürettiği varsayılan kopya kurucu neden tehlikelidir? Yanıtınızda sığ/üye-üye kopyanın işaretçi üyeler için ne anlama geldiğini ve bunun hangi iki somut soruna (paylaşılan durum,

çifte `delete`) yol açabileceğini açıklayın.

**5 • İzleme — Aktarım — Başka Nerede?**

Bu "işaretçi/kaynak tutan bir sınıf kopyalanırken derin kopya gerekir" bilinci `IntTampon`'a özgü değildir. Bir kaynak — bellek, dosya tanıtıcısı, ağ bağlantısı — tutan her sınıf aynı soruyla karşılaşır: bu nesneyi kopyalarken kaynağı paylaştırayım mı, yoksa yeni bir kaynak mı ayırayım? Kitaplık projesinde ileride ham bir kaynak tutacak herhangi bir tip için aynı derin/sığ kopya kararını vereceksiniz. Ünite 3'te bu kararı elle vermek yerine akıllı işaretçilere devredeceksiniz.

## 4.2 Rule of 3, `=default`/`=delete` ve Taşımaya Kısa Bir Bakış

**1 • Ön-öğretim — Neden Önemli?**

Ünite 1'de `IntTampon`'a kopya kurucu ve kopya atama operatörü eklediniz — ama yıkıcısı zaten Hafta 2'den vardı. Bu üç fonksiyonu neden hep birlikte anmak gerekir? Yalnız birini yazıp diğerlerini derleyiciye bırakırsanız, gözden kaçan bir sızıntı ya da yeniden bir çifte `delete` sizi bekliyor olabilir. Öte yandan hiç kaynak tutmayan küçük bir sınıf için bu üçlüyü yazmak gereksiz emek; kopyalanmaması gereken bir sınıf içinse kopyayı büsbütün yasaklamak gerekebilir.

**1 • Ön-öğretim — Ön Bilgi**

Bu üniteye girmeden önce Ünite 1'in kopya kurucu/kopya atama (derin kopya, öz-atama kontrolü) fonksiyonlarını, Hafta 2'nin yıkıcısını (`delete[]`) ve Hafta 3'ün kurucu/yıkıcı biçimsel terimlerini hatırlamanız gerekir. Bu ünite bu üç fonksiyonu tek bir ilke altında toplar ve `=default`/`=delete` sözdizimini ekler.

Önceki ünitede `IntTampon`'a kopya kurucu ve kopya atama operatörünü elle eklediniz. Şimdi bir adım geriye çekilip `IntTampon`'un bugün kaç özel üye fonksiyona elle sahip olduğunu sayın: yıkıcı (Hafta 3'te yazdınız, çünkü sınıf `new[]` ile ayrılmış bir kaynak tutuyor), kopya kurucu ve kopya atama operatörü (bu hafta, Ünite 1'de yazdınız, aynı gerekçeyle). Üçü de — tesadüf değil — aynı köke, `veri` işaretçisinin bir kaynağı sahiplenmesine dayanıyor. Bu gözlem rastgele değildir; C++ camiasında bir isim taşıyan genel bir kurala işaret eder.

### 4.2.1 Rule of 3: neden hep birlikte gelirler?

**Tanım: Rule of 3**

**Rule of 3** şunu söyler: bir sınıfın yıkıcısını, kopya kurucusunu veya kopya atama operatöründen *birini* elle yazmak zorunda kaldıysanız, genellikle *üçünü de* elle yazmak zorundasınızdır.

**Rule of 3:** yıkıcı/kopya kurucu/kopya atamadan biri elle yazılırsa genelde üçünün de yazılması gerektiği ilkesi (*Rule of Three*).

Bu kuralın neden doğru olduğunu soyut bırakmayın, `IntTampon` üzerinden adım adım düşünün. Üç fonksiyonun üçü de, aslında, aynı tek soruyu yanıtlar: "bu sınıf bir kaynağın sahibi mi, ve öyleyse bu kaynağa kimin ne zaman dokunacağını

nasıl garanti ederim?" Yıkıcı bu soruyu "nesne ölürken kaynak nasıl bırakılır" açısından yanıtlar; kopya kurucu "yeni bir nesne var olan birinden doğarken kaynak nasıl paylaşılmadan çoğaltılır" açısından; kopya atama operatörü ise "var olan bir nesnenin kaynağı başka bir nesnenin kaynağıyla değiştirilirken eski kaynak nasıl güvenle bırakılır" açısından. Üçü de aynı zeminde durduğu için, birini elle yazıp ötekileri derleyicinin varsayılanına bırakmak bir tutarsızlık kapısı açar: örneğin `IntTampon`'a yalnızca yıkıcı yazıp kopya kurucuyu varsayılana bıraksaydınız, yıkıcınız `delete[] veri;` çalıştırırken kopya kurucu hâlâ sığ kopya üretirdi — tam olarak Ünite 1'de gördüğünüz çifte `delete` senaryosu kapıda beklerdi. Rule of 3'ün pratik mesajı budur: bu üç fonksiyon bir "set" oluşturur, tek tek değil birlikte düşünülmelidir.

**Analoji**

Rule of 3'ü bir set menü/kombo siparişine benzetebilirsiniz. Normalde bir lokantada standart menüyü (derleyicinin varsayılanını) alırsınız ve her şey hazır gelir. Ama bir gün "hayır, ana yemeği kendim seçeceğim" dediğinizde (yıkıcıyı elle yazdığınızda), garson size yalnızca ana yemeği değil, menünün diğer iki parçasını — çorbayı, tatlıyı (kopya kurucu, kopya atama) — da kendi seçiminize göre tamamlamanızı bekler; standart menüden yalnızca birini özelleştirip diğerlerini "varsayılan" bırakmak, kombo mantığını bozar ve masaya tutarsız bir sipariş gelmesine yol açar. Bir kaynağı özel yönetmeye karar verdiğinizde de aynı kural geçerlidir: üç parçayı birlikte sipariş edersiniz.

### 4.2.2 `=default`: varsayılanı açıkça istemek

Rule of 3, "her sınıf üç fonksiyonu elle yazsın" demez — yalnızca "birini elle yazdıysanız, üçünü de *düşünün*" der. Bazı sınıflar için doğru cevap, üçünü de derleyicinin varsayılanına bırakmaktır; ama bu kararı sessizce (hiçbir şey yazmayarak) değil, *açıkça* belirtmek iyi bir alışkanlıktır. Aşağıdaki küçük `Sayac` sınıfını düşünün:

```
class Sayac {
public:
   explicit Sayac(int baslangic = 0) : deger_(baslangic) {}
   Sayac(const Sayac&) = default;
   Sayac& operator=(const Sayac&) = default;
   int deger() const { return deger_; }
   void artir() { ++deger_; }

private:
   int deger_;
};
```

`Sayac` yalnızca bir `int` tutar — hiçbir kaynağı (işaretçi, dosya tanıtıcısı) sahiplenmez. Böyle bir sınıfta üye üye kopya zaten *doğru* sonucu verir: `deger_`'i kopyalamak, sayının kendisini kopyalamak demektir, paylaşılan bir kaynak riski hiç yoktur. Bu durumda kopya kurucuyu ve kopya atama operatörünü **`=default`** yazımıyla `Sayac(const Sayac&) = default;` biçiminde bildirirsiniz — davranış, hiçbir şey yazmasaydınız derleyicinin zaten üreteceği davranışla birebir aynıdır, ama artık bu tercih kodda *görünür* ve kasıtlıdır: sınıfa bakan başka bir geliştirici, "bu sınıfın kopyalanabilir olması ve varsayılan davranışın yeterli olması bilinçli bir karar" mesajını doğrudan okur.

**`=default`**: derleyiciden bir özel üye fonksiyonun varsayılan sürümünü üretmesini açıkça istemek (*defaulted function*).

### 4.2.3 =delete: bir işlemi tamamen yasaklamak

Bazı sınıflar ise tam tersi bir kararı gerektirir: hiç kopyalanmamaları gerekir. TekilKart sınıfını düşünün — bir izin kartı ya da bilet gibi, kopyası anlam taşımayan, tekil bir kaynağı temsil eder:

```
21 class TekilKart {
22 public:
23    explicit TekilKart(int numara) : numara_(numara) {}
24    TekilKart(const TekilKart&) = delete;
25    TekilKart& operator=(const TekilKart&) = delete;
26    int numara() const { return numara_; }
27
28 private:
29    int numara_;
30 };
```

**=delete**: bir işlemi (ör. kopyalamayı) tamamen yasaklayıp derleme hatasına çeviren bildirim (*deleted function*).

Burada kopya kurucu ve kopya atama operatörü **=delete** ile işaretlenmiştir: `TekilKart(const TekilKart&) = delete;`. Bu satırın anlamı "derleyici bu işlem için hiçbir sürüm üretmesin, üstelik biri bu işlemi çağırmayı denerse derleme *başarısız olsun*" demektir. Somut sonucu şudur: `araclar::TekilKart k2 = k1;` gibi bir satır yazmayı denerseniz, program hiç çalışmaz — derleyici sizi daha derleme aşamasında durdurur. Bu, `=default`'tan tamamen farklı bir niyettir: `=default` "varsayılan davranış zaten doğru, onu istiyorum" derken, `=delete` "bu işlem bu sınıf için anlamsız, kimse yapmasın" der.

**Analoji**

`=default` ile `=delete`'i bir başvuru formundaki iki kutuya benzetebilirsiniz. Formda bir alan için "standart cevabı kullan" diye işaretlenebilecek bir kutu vardır (`=default`) — formu dolduran, bu alan için özel bir şey istemediğini açıkça belirtir. Bir de "bu seçenek kapalıdır, işaretlenemez" diye damgalanmış bir alan vardır (`=delete`) — formu dolduran kişi o kutuyu hiçbir biçimde işaretleyemez, denerse form geçersiz sayılır. Derleyici de tam olarak bu formu okuyan görevlidir: `Sayac` formunda "standart kopya yeterli" kutusu işaretlidir, `TekilKart` formunda ise "kopyalama" alanı baştan kapatılmıştır.

Bu iki sınıfı gerçek kullanımda görün:

```
7    std::cout << "-- Sayac (=default) --\n";
8    araclar::Sayac s1(5);
9    araclar::Sayac s2 = s1; // kopya kurucu = default, sorunsuz calisir
10   s2.artir();
11   std::cout << "s1.deger()=" << s1.deger() << " s2.deger()=" << s2.deger()
         << "\n";
12
13   std::cout << "-- TekilKart (=delete) --\n";
14   araclar::TekilKart k1(42);
15   std::cout << "k1.numara()=" << k1.numara() << "\n";
16   // Asagidaki satir KASITLI OLARAK yorumda birakilmistir; acilirsa
17   // derleme hatasi verir (kopya kurucu = delete edildigi icin):
18   // araclar::TekilKart k2 = k1;
```

`Sayac s2 = s1;` satırı sorunsuz derlenir ve çalışır — `=default` işaretli kopya kurucu devreye girer, `s2` kendi bağımsız `deger_` kopyasını alır. Yorum satırındaki `araclar::TekilKart k2 = k1;` ise *kasıtlı olarak* açılmamıştır; açılsaydı, kopya kurucu `=delete` edildiği için program derlenmezdi.

### 4.2.4 Rule of 0 ve Rule of 5: isim düzeyinde iki komşu kural

Rule of 3'ün doğal bir uzantısı vardır: eğer sınıfınız hiç ham kaynak tutmuyorsa — yalnızca kendi kaynak yönetimini zaten üstlenen üyeler (`std::string`, STL kapları, birazdan göreceğiniz akıllı işaretçiler) kullanıyorsa — o zaman hiçbir özel üye fonksiyonu elle yazmanıza hiç gerek kalmaz; derleyicinin ürettiği memberwise kopya, atama ve yıkım zaten doğru sonucu verir, çünkü her üye kendi kaynağını kendi doğru biçimde yönetir. Bu ilkeye **Rule of 0** denir; Ünite 3'teki `AkilliTampon` sınıfı bu ilkenin somut, çalışan bir örneği olacak.

**Rule of 0**: kaynak yönetimini üstlenen hazır üyeler kullanılırsa hiçbir özel fonksiyonun elle yazılmasına gerek kalmaması ilkesi (*Rule of Zero*).

Son olarak bir kural daha isim düzeyinde anılmayı hak eder: C++11'den itibaren, kopyalamanın yanına bir de *taşıma* kavramı eklendi ve bu iki yeni özel üye fonksiyonu — taşıma kurucusu ve taşıma atama operatörü — getirdi; yıkıcı, kopya kurucu, kopya atama, taşıma kurucusu, taşıma atama birlikte anıldığında bu beşliye *Rule of 5* denir. Bu haftanın kapsamı Rule of 5'in derinliğine girmez — yalnızca adını tanımanız yeterlidir; taşıma kurucusunu bu hafta elle yazmayacaksınız.

> **İpucu**
>
> **Kısa bir bakış.** `std::move(x)` çağrısı, bir nesnenin kaynağını kopyalamadan başka bir nesneye devretme niyetini taşıyan **taşıma (move) semantiği** fikrinin aracıdır: `x`'i fiilen hiçbir şey "taşımaz" — yalnızca derleyiciye "bu değeri artık bir rvalue referansı gibi ele alabilirsin" diye işaretler. `IntTampon`'un bu haftaki hâlinde kullanıcı tanımlı bir taşıma kurucusu *yazılmadığı* için, `std::move` kullanmak yine de derlenir ama — taşıma kurucusu bulunmadığından — *kopya kurucuya düşer*: güvenlidir, ama gereksiz bir derin kopya yapılmış olur, yani verimsizdir. Aşağıdaki kesiti izleyin (`c`, Ünite 1'deki atama demosundan gelen aynı `IntTampon` nesnesidir):
>
> ```
> 38  std::cout << "-- tasima onizleme --\n";
> 39  araclar::IntTampon d(std::move(c));
> 40  // Not: IntTampon icin kullanici tanimli bir tasima kurucusu YAZILMADI
>         ;
> 41  // std::move(c) yine de derlenir ama burada -- tasima kurucusu
> 42  // olmadigi icin -- kopya kurucuya duser (guvenli, fakat gereksiz bir
> 43  // derin kopya yapilir; verimsiz). c bu yuzden hala gecerli kalir.
> 44  std::cout << "d.veri[1]=" << d.veri[1] << "\n";
> ```
>
> `IntTampon d(std::move(c));` satırı derlenir ve çalışır, ama arka planda `c`'nin kopya kurucusu çağrılır — `d`, `c`'nin verisinin yeni bir derin kopyasını alır, `c` de hâlâ geçerli kalır (taşınmamış, kopyalanmıştır). Bu haftada taşıma kurucusunu elle yazmayacaksınız; fikri yalnızca şu düzeyde tutun: "taşımak, kopyalamaktan farklı bir *niyettir*" — taşımanın gerçek kazancı (kopya yapmadan sahiplik devretmek), Ünite 3'te `unique_ptr`'ın taşımayı zorunlu kılmasıyla somutlaşacak.

**taşıma (move) semantiği**: bir nesnenin kaynağını kopyalamadan başka bir nesneye devretme niyeti (*move semantics*); bu hafta yalnızca kısa/opsiyonel bir değini düzeyinde ele alınır.

Bu ünitede gördüğünüz fikir kısaca şudur: yıkıcı, kopya kurucu ve kopya atama operatörü aynı soruyu — bu sınıf bir kaynağın sahibi mi — yanıtladığı için Rule of 3 bu üçünü bir arada düşünmenizi ister; `=default` bu üç fonksiyondan varsayılan davranışın zaten doğru olduğu durumlarda niyetinizi açıkça belirtmenizi, `=delete` ise bir işlemi tamamen yasaklamak istediğinizde bunu derleme hatasına çevirmenizi sağlar. Rule of 0, hiç kaynak tutmayan ya da kaynağını hazır üyelere devreden sınıflarda bu üç fonksiyonu hiç yazmamanın en temiz yol olduğunu söyler — bir sonraki ünitede bunu `unique_ptr` ile somut olarak göreceksiniz.

**3 • Katılım — Şimdi Siz Deneyin**

Yalnız bir `int` üye tutan küçük bir `Sayac` sınıfı ile, bir izni temsil eden ve asla kopyalanmaması gereken bir `TekilKart` sınıfı düşünün. (a) `Sayac`'ın kopya kurucusunu/atamasını nasıl belirtirsiniz? (b) `TekilKart`'ı kopyalamaya çalışan `TekilKart k2 = k1;` satırı için ne yaparsınız ve o satır ne olur?

---

**Geribildirim.** (a) `Sayac` kaynak tutmaz; üye üye kopya zaten doğrudur, bu yüzden kopya kurucu/atamayı `=default` ile açıkça "varsayılanı üret" diye belirtirsiniz — elle Rule of 3 uygulamaya gerek yoktur. (b) `TekilKart`'ın kopya kurucu/atamasını `=delete` edersiniz; o zaman `TekilKart k2 = k1;` bir *derleme* hatası verir ("use of deleted function") — istenmeyen kopya çalışma zamanına kalmadan, daha derlemede engellenir. Bu `=delete` fikri, Ünite 3'te `unique_ptr`'ın kopyalanamaz oluşunun habercisidir.

**Kendinizi Sınayın 4.2**

Rule of 3 ne der ve `TekilKart` örneğinde kopya kurucu ile kopya atama operatörünü neden `=delete` ile tamamen yasaklarsınız? Yanıtınızda üç özel fonksiyonun ortak sorusunu ve bazı kaynakların hiç kopyalanmaması gerektiği fikrini ele alın.

**5 • İzleme — Aktarım — Başka Nerede?**

"Kaynağın sahipliği tek bir soru; üç fonksiyon onu birlikte cevaplar" düşüncesi kaynak yöneten her sınıf için geçerlidir. `=delete` ile bir işlemi tamamen kapatma tekniği ise kopyalama dışında da işe yarar — kimi tipler taşınabilir ama kopyalanamaz olmalıdır. Kitaplık projesinde tek bir fiziksel kopyası olan bir ödünç kaydını (aynı anda iki rafta görünmemesi gereken) modellerken de "kopyalamayı yasakla" kararını buradaki `=delete` ile verirsiniz.

## 4.3 Akıllı İşaretçiler: `unique_ptr` ve `shared_ptr`

**1 • Ön-öğretim — Neden Önemli?**

Ünite 1–2'de `IntTampon`'u Rule of 3 ile "elle" doğru hâle getirdiniz — ama üç fonksiyonun her birini kusursuz yazma sorumluluğu hâlâ sizde; tek bir unutulmuş öz-atama kontrolü ya da eksik `new[]` sızıntı demektir. Ya bu sorumluluğu bütünüyle devredebilseydiniz? Akıllı işaretçiler tam da bunu yapar: kaynağı ne zaman bırakacağını kendileri bilir, siz `new`/`delete` yazmazsınız.

**1 • Ön-öğretim — Ön Bilgi**

Bu üniteye girmeden önce Ünite 1'in derin kopya/kaynak sahipliği, Ünite 2'nin `=delete` (kopyalamayı yasaklama) ve Rule of 0 fikirlerini, ayrıca Hafta 2'nin `new[]`/`delete[]` ile elle bellek yönetimini hatırlamanız gerekir. Bu ünite elle yönetimi kütüphaneye (`unique_ptr`/`shared_ptr`) devreder.

Önceki iki ünitede `IntTampon`'u Rule of 3 ile *elle* doğru hâle getirdiniz: bir yı-

kıcı, bir kopya kurucu, bir kopya atama operatörü — üçü de kaynağı derin kopyalayan, öz-atamayı kontrol eden, tutarlı bir üçlü. Ama bu çözümün hâlâ bir zayıf noktası var: doğruluğu tamamen *size* bağlıdır. Üç fonksiyonun her birini doğru yazmak, öz-atama kontrolünü unutmamak, her `new[]`'e karşılık bir `delete[]` yazmak — bunların hepsi programcının sorumluluğunda kalır, ve programcının disiplinine bırakılan her kuralın er ya da geç unutulabileceğini Ünite 1'de gördünüz. Akıllı işaretçiler bu sorumluluğu programcıdan alıp *kütüphaneye* devreder: kaynağı hâlâ öbekte tutarsınız, ama onu ne zaman bırakacağınızı elle yazmazsınız.

### 4.3.1 `unique_ptr`: tekil sahiplik

**Tanım: unique_ptr**

**unique_ptr**: bir kaynağı tekil ve kopyalanamaz biçimde sahiplenen, yalnız taşınabilen akıllı işaretçi (*unique pointer*).

**`unique_ptr`** bir kaynağı — öbekte ayrılmış bir nesne ya da diziyi — kendi içinde tutan bir sınıf şablonudur; kaynağı aynı anda yalnızca *bir* `unique_ptr` sahiplenebilir. `unique_ptr` kapsam dışına çıktığında, yıkıcısı otomatik çalışıp tuttuğu kaynağı kendisi bırakır — tıpkı `IntTampon`'un kendi yıkıcısının `delete[]` çalıştırması gibi, ama artık bu yıkıcıyı siz yazmazsınız, kütüphane sizin için zaten yazmıştır.

Tekil sahipliğin somut sonucu, `unique_ptr`'ın *kopyalanamaz* olmasıdır: kopya kurucusu ve kopya atama operatörü örtük olarak `=delete` edilmiştir — tam olarak bir önceki ünitede `TekilKart` örneğinde elle yazdığınız desenin aynısı, ama bu kez kütüphanenin kendisi tarafından, sizin hiçbir şey yazmanıza gerek kalmadan uygulanır. Bir `unique_ptr`'ı bir yerden bir yere *devretmek* isterseniz, tek yol `std::move` ile *taşımaktır*: kaynak, eski sahipten yeni sahibe geçer, eski sahip artık boş (kaynağı olmayan) bir duruma düşer — aynı kaynağın aynı anda iki sahibi asla olmaz. Bir `unique_ptr` oluştururken çıplak `new` yazmazsınız; bunun yerine, çıplak `new` yazmadan akıllı işaretçi oluşturan **`make_unique` ve `make_shared`** yardımcı fabrika fonksiyonlarından `std::make_unique<T>(...)`'ı kullanırsınız — bu fonksiyon kaynağı ayırır ve onu doğrudan bir `unique_ptr` içine sarar, aradan çıplak bir `new`/işaretçi çifti hiç geçmez.

**make_unique ve make_shared**: çıplak *new* yazmadan akıllı işaretçi oluşturan yardımcı fabrika fonksiyonları (*factory functions*).

**Analoji**

`unique_ptr`'ı devredilebilir ama çoğaltılamaz tek bir bagaj fişine benzetebilirsiniz. Fiş elden ele verilebilir — birine uzatırsınız, o andan itibaren bagaj (kaynak) onundur, sizde artık fiş kalmaz — ama fotokopisi çekilemez: aynı anda iki kişide aynı fişin geçerli bir kopyası olamaz. Fiş kimdeyse bagajın sahibi odur; fişi kaybeden ya da elden çıkaran kişi artık bagaja erişemez. `std::move` ile bir `unique_ptr`'ı taşımak, tam olarak bu fişi elden ele uzatmaktır — kopyalamak değil, devretmektir.

Bu ilkeleri `IntTampon`'un akıllı işaretçili karşılığında görün: `AkilliTampon`.

```
8  class AkilliTampon {
9  public:
10     explicit AkilliTampon(int n);
11     int boyut() const { return boyut_; }
12     int& operator[](int i) { return veri_[i]; }
13     int operator[](int i) const { return veri_[i]; }
14
15 private:
```

```
16    std::unique_ptr<int[]> veri_;
17    int boyut_;
18 };
```

Bu sınıf gövdesini `IntTampon` ile yan yana koyun: `IntTampon` ham bir `int* veri` tutuyordu ve kendi kurucusunu, yıkıcısını, kopya kurucusunu, kopya atama operatörünü elle yazmanız gerekiyordu. `AkilliTampon` ise `std::unique_ptr<int[]>` tipinde tek bir üye tutar — ve dikkat edin, sınıf gövdesinde *hiçbir* kullanıcı tanımlı yıkıcı, kopya kurucu ya da kopya atama operatörü yoktur. Bu, önceki ünitede tanıdığınız Rule of 0'ın birebir uygulamasıdır: kaynak yönetimi tamamen `veri_` üyesine, yani `unique_ptr`'ın kendisine devredilmiştir; `AkilliTampon`'un kendi kurucusunun tek işi `veri_`'yi doğru büyüklükte kurmaktır:

```
8  AkilliTampon::AkilliTampon(int n)
9     : veri_(std::make_unique<int[]>(n)), boyut_(n) {
10    std::cout << "akilli tampon olusturuldu (boyut=" << boyut_ << ")\n";
11 }
```

`std::make_unique<int[]>(n)` çağrısı `n` elemanlık bir diziyi öbekte ayırır ve doğrudan bir `unique_ptr<int[]>` olarak döndürür — çıplak `new int[n]` hiçbir yerde yazılmaz. Şimdi `AkilliTampon` kapsam dışına çıktığında ne olur diye sorun: `AkilliTampon`'un kendi yıkıcısı yoktur, ama derleyicinin ürettiği varsayılan yıkıcı her üyenin kendi yıkıcısını sırayla çağırır — `veri_` bir `unique_ptr` olduğu için onun yıkıcısı çalışır, o da tuttuğu diziyi kendiliğinden bırakır. Sonuç, `IntTampon`'un elle yazdığınız yıkıcısıyla birebir aynıdır, ama siz tek bir `delete[]` satırı bile yazmadınız.

Aynı mantık kopyalanabilirlik için de geçerlidir: `unique_ptr` kopyalanamaz olduğu için, onu üye olarak tutan `AkilliTampon` da *kendiliğinden* kopyalanamaz hâle gelir — derleyici, `AkilliTampon` için bir kopya kurucu üretmeye çalıştığında, bu kurucunun `veri_`'yi de kopyalaması gerektiğini görür, ama `veri_`'nin kendi kopya kurucusu `=delete` olduğu için üretilen kopya kurucu da örtük olarak kullanılamaz hâle gelir. Bu, Ünite 2'nin `=delete` fikrinin burada *otomatik* olarak, sizin hiçbir şey yazmanıza gerek kalmadan gerçekleşmesidir. Gerçek kullanımda görün:

```
10    std::cout << "-- unique_ptr demo --\n";
11    araclar::AkilliTampon a(5);
12    for (int i = 0; i < a.boyut(); ++i) {
13       a[i] = i * 10;
14    }
15    araclar::AkilliTampon b = std::move(a); // sahiplik b'ye tasinir
16    std::cout << "b: ";
17    for (int i = 0; i < b.boyut(); ++i) {
18       std::cout << b[i] << " ";
19    }
20    std::cout << "\n";
21    // Asagidaki satir DERLENMEZ: unique_ptr uyesi kopyalanamaz oldugu
22    // icin AkilliTampon de kendiliginden kopyalanamaz hale gelir:
23    // araclar::AkilliTampon c = b;
```

`AkilliTampon b = std::move(a);` satırını adım adım okuyun: `a` beş elemanla kurulur ve doldurulur; `std::move(a)` `a`'yı bir rvalue referansı olarak işaretler; `AkilliTampon`'un (derleyici üretimi) taşıma kurucusu devreye girer ve `a`'nın `veri_`'sindeki `unique_ptr`'ı `b`'nin `veri_`'sine *taşır* — kaynağın sahipliği `a`'dan `b`'ye geçer, `a` artık boş bir `unique_ptr` tutar. `b` üzerinden yazdırılan değerler doğru çıkar, çünkü kaynağın kendisi hiç kopyalanmadı, yalnızca sahibi değişti. Yorum satırındaki `araclar::AkilliTampon c = b;` ise *kasıtlı olarak* açılmamıştır — açılsaydı derlenmezdi, çünkü az önce açıkladığınız gibi `AkilliTampon` kendiliğinden

kopyalanamaz.

### 4.3.2 shared_ptr: paylaşılan sahiplik

unique_ptr'ın tekil sahiplik kuralı her senaryoya uymaz — bazen birden çok yerin, aynı anda, aynı kaynağı gerçekten *paylaşarak* yaşatması gerekir. Bu ihtiyaç için farklı bir araç vardır:

**shared_ptr**: bir kaynağı referans sayımıyla birden çok sahip arasında paylaştıran akıllı işaretçi (*shared pointer*).

**Tanım: shared_ptr**

**shared_ptr** aynı kaynağı gösteren birden çok shared_ptr nesnesinin var olmasına izin verir; her biri o kaynağın *eş sahibidir*. Bunu mümkün kılan şey dahili bir *referans sayımı* mekanizmasıdır (adını birazdan göreceksiniz): her yeni shared_ptr kopyalandığında sayaç bir artar, bir shared_ptr kapsam dışına çıktığında sayaç bir azalır; sayaç sıfıra düştüğü — yani *son* sahip de kapsam dışına çıktığı — anda kaynak otomatik bırakılır. Sayacın o anki değerini use_count() ile okuyabilirsiniz.

unique_ptr gibi shared_ptr için de çıplak new yazmazsınız; std::make_shared kullanırsınız. Bunun yalnızca bir alışkanlık değil, somut bir verimlilik farkı olduğunu bilin: make_shared nesnenin kendisini *ve* referans sayacını tek bir öbek tahsisinde bir arada ayırır; new T(...) yazıp sonucu ayrı bir shared_ptr'a sarmak ise iki ayrı tahsis gerektirir (biri nesne için, biri sayaç için) — make_shared bu ikisini birleştirerek daha verimli çalışır.

**Analoji**

shared_ptr'ı ortak kullanılan bir aile aboneliğine benzetebilirsiniz. Abonelik (kaynak) tek bir yerde durur, ama kaç kişi aboneliği kullanıyorsa sayaç o kadardır. Ailenin yeni bir üyesi aboneliğe katıldığında sayaç artar (bir shared_ptr kopyalanır); biri ayrıldığında sayaç düşer (bir shared_ptr kapsam dışına çıkar). Kaç kişi kullanıyorsa abonelik o kadar sürer — ama *son* kişi de ayrılınca abonelik tamamen kapanır, tıpkı sayaç sıfıra indiğinde kaynağın otomatik bırakılması gibi.

Şimdi bu davranışı gerçek bir sayaç üzerinden izleyin:

```
27  std::cout << "-- shared_ptr demo --\n";
28  auto p1 = std::make_shared<araclar::IntTampon>(5);
29  std::cout << "p1.use_count()=" << p1.use_count() << "\n";
30  {
31     auto p2 = p1; // paylasilan sahiplik, sayac artar
32     std::cout << "p1.use_count()=" << p1.use_count()
33               << " p2.use_count()=" << p2.use_count() << "\n";
34  } // p2 kapsam disina cikar, sayac azalir
35  std::cout << "p1.use_count()=" << p1.use_count() << "\n";
```

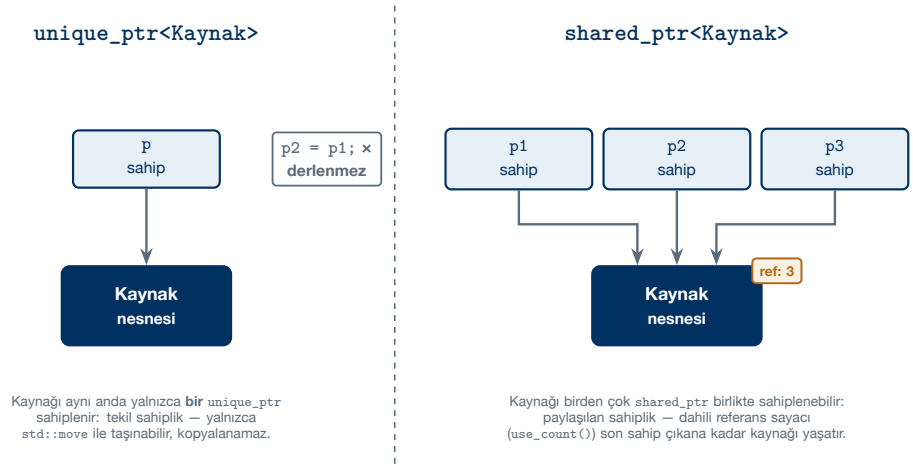
Bu satırları sırasıyla okuyun: p1 tek başına bir araclar::IntTampon kaynağını sahiplenir, p1.use_count() 1 basar. İç kapsamda auto p2 = p1; çalıştığında yeni bir kaynak *oluşturulmaz* — p2 yalnızca p1'in gösterdiği *aynı* kaynağa eş sahip olur, dahili sayaç bire artar; hem p1.use_count() hem p2.use_count() artık 2 basar. Bu sayaç, **referans sayımı** adı verilen mekanizmanın ta kendisidir. İç kapsam kapandığında p2 kapsam dışına çıkar, sayaç yeniden bir azalır; kapsam dışına çıktıktan sonraki p1.use_count() tekrar 1'e döner — kaynağın kendisi henüz bırakılmamıştır, çünkü p1 hâlâ hayattadır ve sayaç sıfıra inmemiştir.

**referans sayımı**: shared_ptr'ın kaç sahip olduğunu izleyip son sahip çıkınca kaynağı bırakmasını sağlayan sayaç (*reference counting*).

### 4.3.3 Derin kopya mı, paylaşılan sahiplik mi: iki farklı niyet

Ünite 1'deki `IntTampon b = a;` ile buradaki `auto p2 = p1;` satırlarını yan yana koyun — ikisi de görünüşte "bir kopya" üretir, ama kastettikleri şey taban tabana zıttır. `IntTampon b = a;` *nesne düzeyinde* bir derin kopyadır: `a` ile `b` kopyadan sonra birbirinden tamamen bağımsız iki kaynağa sahiptir, biri değişse öteki etkilenmez. `auto p2 = p1;` ise (burada `p1` ve `p2` birer `shared_ptr<IntTampon>` olduğunda) tam tersini yapar: `p1` ile `p2` *aynı* `IntTampon` nesnesini paylaşır, kopyalanan yalnızca "sahiplik" bilgisidir, kaynağın kendisi değil. İki farklı niyet, iki farklı araç: bağımsız bir kopya istiyorsanız derin kopya (kopya kurucu/atama ya da doğrudan nesne kopyası); aynı kaynağı birden çok yerin *paylaşarak* yaşatması gerekiyorsa `shared_ptr`.

Şekil 4.2 bu iki modeli yan yana özetliyor: solda `unique_ptr`'ın tekil, taşınabilir ama kopyalanamaz sahipliği, sağda `shared_ptr`'ın referans sayımlı, birden çok sahibe açık paylaşılan sahipliği.




Şekil 4.2: `unique_ptr` (solda): tek sahip, taşınabilir, kopyalanamaz. `shared_ptr` (sağda): birden çok sahip, referans sayımlı.

> **İpucu**
>
> Hangi durumda hangisini seçeceğinize karar vermek için basit bir kural izleyin: varsayılan tercihiniz her zaman `unique_ptr` olsun — en ucuz (sayaç tutmaz), en açık (tek sahip, kim sahiplendiği kodda net) seçenektir. `shared_ptr`'ı yalnızca gerçekten birden çok sahibin *aynı anda* kaynağı yaşatması gerektiği durumlarda — örneğin aynı veri birden çok bağımsız bileşen tarafından, hangisinin daha uzun yaşayacağı önceden belli olmadan kullanılacaksa — tercih edin. Bu haftanın kapsamı dışında kalan bir üçüncü akıllı işaretçi türü daha vardır (`weak_ptr`, döngüsel referans problemini çözmek için) — şimdilik yalnızca adını bilmeniz yeterli, ileri bir konudur.

Bu ünitede gördüğünüz fikir kısaca şudur: akıllı işaretçiler, önceki iki ünitede elle taşıdığınız Rule of 3 sorumluluğunu kütüphaneye devreder. `unique_ptr` tekil, kopyalanamaz ama taşınabilir bir sahiplik sunar ve `AkilliTampon` örneğinde gördüğünüz gibi Rule of 0'ı otomatik olarak mümkün kılar; `shared_ptr` ise referans sayımıyla paylaşılan bir sahiplik sunar. İkisi de `make_unique`/`make_shared` ile, çıplak `new` yazmadan oluşturulur. Gelecek ünitede bu araçları, kütüphanenin ilk bağlı veri yapısını — `IntBagliListe`'yi — inşa etmek için kullanacaksınız.

**3 • Katılım** **Şimdi Siz Deneyin**

`AkilliTampon`, `std::unique_ptr<int[]>` üyeli, hiçbir kullanıcı tanımlı kopya/yıkıcı içermeyen (Rule of 0) bir sınıf olsun. `AkilliTampon b = std::move(a);` satırı ile `AkilliTampon c = b;` satırından hangisi derlenir, hangisi derlenmez, neden?

---

**Geribildirim.** `AkilliTampon b = std::move(a);` *derlenir*: taşıma, sahipliği `a`'dan `b`'ye devreder (`a` artık boştur). `AkilliTampon c = b;` *derlenmez*: `unique_ptr` üyesi kopyalanamaz (kopya kurucusu örtük `=delete`), dolayısıyla onu içeren `AkilliTampon`'un kopya kurucusu da örtük olarak `=delete` sayılır ("use of deleted function"). Yani Ünite 2'de `TekilKart` için *elle* yazdığınız `=delete`, burada `unique_ptr` sayesinde *kendiliğinden* gerçekleşir. `shared_ptr` kullansaydınız `auto p2 = p1;` kopyalanır ve `use_count()` 1'den 2'ye çıkardı.

**Kendinizi Sınayın 4.3**

`unique_ptr` ile `shared_ptr` arasındaki temel fark nedir, hangi durumda hangisini seçersiniz? Yanıtınızda tekil/kopyalanamaz+taşınabilir sahiplik ile paylaşılan/referans-sayımlı sahiplik ayrımını ve varsayılan tercihin neden `unique_ptr` olduğunu ele alın.

**5 • İzleme** **Aktarım — Başka Nerede?**

`unique_ptr`/`shared_ptr` yalnız `IntTampon` benzeri tamponlar için değil, ham `new`/`delete` yazdığınız her yerde geçerlidir; varsayılan tercihiniz `unique_ptr` olsun, gerçekten paylaşım gerekince `shared_ptr`. Ünite 4'te tam bunu kullanacaksınız: bir bağlı listenin düğümlerini `unique_ptr` ile bağlayacak, hiç elle `delete` yazmayacaksınız. Kitaplık projesinde ileride birden çok raf/kaydın aynı `Kitap` nesnesini paylaşması gerektiğinde `shared_ptr`, tek sahipli ilişkilerde `unique_ptr` doğal seçiminiz olur.

## 4.4 Uygulama: `IntBagliListe` — Kütüphanenin İlk Bağlı Yapısı

**1 • Ön-öğretim** **Neden Önemli?**

Şimdiye dek tek bir kaynak tutan sınıflarla çalıştınız; ama gerçek veri yapıları birbirine bağlı onlarca parçadan oluşur. Kütüphaneye ilk bağlı yapıyı — bir `IntBagliListe` — eklerken her düğümün bir sonrakini "sahiplenmesi" gerekir. Bu zinciri elle `new`/`delete` ile kurmak, her düğümü doğru sırada silmeyi hatırlamak demektir; oysa `unique_ptr` ile zinciri kurarsanız, listeyi silmek için tek bir satır bile yazmanıza gerek kalmaz.

**1 • Ön-öğretim** **Ön Bilgi**

Bu üniteye girmeden önce Ünite 3'ün `unique_ptr`'ını (tekil sahiplik, `make_unique`, `std::move` ile taşıma), gözlemci ham işaretçi fikrini ve Rule

of 0'ı (yıkıcı yazmama) hatırlamanız gerekir. Hafta 3'ün iç içe/üye sınıf yapısı ve önceki derslerden bağlı liste sezgisi (düğüm + `sonraki` bağlantısı) da işinize yarayacak; buradaki yenilik, `sonraki`'nin bir `unique_ptr` olmasıdır.

Önceki iki ünitede tek bir tampon — `IntTampon`, sonra `AkilliTampon` — üzerinde çalıştınız: tek bir bellek bloğu, sabit sayıda eleman, tek bir sahiplik ilişkisi. Bu ünitede kütüphaneye tamamen yeni bir tür yapı ekliyorsunuz: `araclar::IntBagliListe`, tek yönlü, `int` tutan somut tipli bir bağlı liste — kütüphanenin ilk *bağlı* veri yapısı. "Somut tipli" vurgusu bilinçlidir: bu liste yalnızca `int` tutar, herhangi bir tip için genel (generic) bir liste değildir — şablonlarla genelleştirme Hafta 9'un konusudur, bu hafta kapsam dışıdır. Burada asıl önemli olan, bir zincirin her halkasını akıllı işaretçilerle nasıl birbirine bağlayacağınızı görmenizdir.

### 4.4.1 Düğüm: zincirin bir halkası

**düğüm**: bağlı bir yapıda bir veri ve bir sonrakine bağlantı tutan birim (*node*).

Bağlı bir listenin temel birimine **düğüm** denir. `IntBagliListe`'nin iç içe tanımlı `Dugum` tipine bakın:

```
18    struct Dugum {
19        int deger;
20        std::unique_ptr<Dugum> sonraki;
21    };
```

İki alanı var: `deger`, düğümün taşıdığı sayı; `sonraki` ise zincirdeki bir sonraki düğümü *sahiplenen* bir `std::unique_ptr<Dugum>`. Bu son ayrıntıyı — `sonraki`'nin ham bir `Dugum*` değil, bir `unique_ptr<Dugum>` olması — gözden kaçırmayın, çünkü Ünite 3'ün tekil sahiplik fikrinin burada neden bu kadar doğal oturduğunu açıklar. Bir zincirde her düğümü *yalnızca bir* önceki düğüm sahiplenir — düğüm listenin başındaysa onu listenin kendisi sahiplenir, değilse onu ondan önceki düğüm sahiplenir; hiçbir düğüm aynı anda iki farklı yerden sahiplenilmez. Bu, tam olarak `unique_ptr`'ın modellediği ilişkidir: tekil, kopyalanamaz, yalnızca taşınabilir bir sahiplik.

**Analoji**

Bağlı listeyi bir define avında art arda gelen ipucu kağıtlarına benzetebilirsiniz. Her ipucu bir sonraki ipucunun yerini gösterir; son ipucunda "define burada, başka ipucu yok" yazar — bir sonraki işaretçinin `nullptr` olması gibi. Kritik nokta şu: her ipucu yalnız *bir* sonraki ipucunu "sahiplenir" — ipucu zincirinde kimse aynı kağıda iki yerden aynı anda sahip değildir; bir ipucu elden ele geçebilir (taşınabilir), ama fotokopisi anlamsızdır (kopyalanamaz). Bu, Ünite 3'te gördüğünüz `unique_ptr` zihniyetiyle birebir uyum içindedir.

Listenin kendisi de aynı ilkeyi tek bir üyede taşır:

```
25    std::unique_ptr<Dugum> bas_;
```

`bas_`, zincirin ilk düğümünü sahiplenen tek üyedir. Liste boşken `bas_`, bir `unique_ptr`'ın varsayılan boş durumundadır, yani `nullptr`'a eşdeğer olur. Listenin genel arayüzü ise kısa ve sadedir:

```
10    IntBagliListe() = default;
11    void ekleBasa(int deger);
12    void yazdir() const;
13    bool bosMu() const { return bas_ == nullptr; }
```

Dört bildirimden ilki — `IntBagliListe() = default;` — Ünite 2'den tanıdık bir sözdizimidir: hiçbir özel ilklemeye gerek yoktur, çünkü `bas_` zaten kendi varsayılan (boş) durumunda başlar. Şu ana kadar hiçbir yerde bir yıkıcı yazmadığınıza dikkat edin — ve *yazmayacaksınız* da. Bunun nedenini bir sonraki paragrafta göreceksiniz.

### 4.4.2 Yıkıcıyı neden hiç yazmazsınız?

`IntBagliListe` kaynağı olmayan bir sınıf gibi *görünmeyebilir* — sonuçta öbekte düğümler tutuyor — ama Rule of 0 burada da tam olarak geçerlidir, çünkü tek üyesi (`bas_`) kendi kaynağını kendi yöneten bir `unique_ptr`'dır. `IntBagliListe` kapsam dışına çıktığında derleyicinin ürettiği varsayılan yıkıcı `bas_`'ın kendi yıkıcısını çağırır; `bas_`'ın yıkıcısı ise kendi `Dugum`'unu bırakmadan önce, o `Dugum`'un *kendi* `sonraki` üyesinin yıkıcısını çağırır — bu da kendi `sonraki`'sininkini çağırır, ve bu zincir son düğüme (`sonraki`'si `nullptr` olan düğüme) kadar ardışık olarak devam eder. Sonuç: tek bir `bas_`'ı yok etmek, zincirdeki *her* düğümü, birer birer, otomatik olarak yok eder — siz hiçbir döngü ya da elle `delete` yazmadan.

**Ortak Hata / Tuzak**

Bu ardışık/zincirleme yıkımın bir teorik tuzağı olduğunu bilmekte fayda var (bu haftanın derinliğinde ele almanız zorunlu değildir, ama farkında olmanız iyidir): her `Dugum`'un yıkıcısı kendi `sonraki`'sinin yıkıcısını *çağırarak* çalışır — bu, fonksiyon çağrı yığınında iç içe geçen bir zincirdir. Çok uzun bir listede (örneğin yüz binlerce düğüm) bu iç içe çağrı zinciri, teorik olarak çağrı yığınını (*call stack*) taşırabilir. Bu haftanın örneklerindeki liste boyutları bu riskin çok uzağındadır; yalnızca "otomatik yıkım her zaman bedelsiz değildir" fikrini aklınızda bir tuzak olarak tutmanız yeterlidir.

### 4.4.3 `ekleBasa`: zinciri kırmadan kaydırmak

**bağlı liste**: düğümlerin birbirine (genelde tek yönlü) bağlantılarla zincirlendiği veri yapısı (*linked list*).

Bir **bağlı liste** inşa etmenin en temel işlemi, başa yeni bir düğüm eklemektir. Gövdeyi adım adım izleyin:

```
void IntBagliListe::ekleBasa(int deger) {
    auto yeniDugum = std::make_unique<Dugum>();
    yeniDugum->deger = deger;
    yeniDugum->sonraki = std::move(bas_); // eski bas, yeni dugumun sonrakisi
        olur
    bas_ = std::move(yeniDugum); // sahiplik yeni dugume kayar
}
```

Bu üç satırı sırasıyla okuyun; sıra burada rastgele değil, *zorunludur*. **Birinci satır**, `std::make_unique<Dugum>()` ile öbekte yepyeni, boş bir `Dugum` ayırır ve onu yerel bir `unique_ptr` olan `yeniDugum`'a bağlar — bu noktada `yeniDugum` zincirin dışında, tek başına duran bir düğümdür. **İkinci satır**, `yeniDugum->sonraki = std::move(bas_);`, en kritik adımdır: `bas_`'ın o an sahip olduğu — listenin *eski* başı olan — düğümü, `std::move` ile `yeniDugum`'un `sonraki` üyesine *taşır*. Bu satırdan sonra eski baş düğüm artık `yeniDugum`'un altında bir halka olarak durur; `bas_` ise (taşıma sonrası `unique_ptr`'ların davranışı gereği) boş kalır. **Üçüncü satır**, `bas_ = std::move(...)`, sahipliği bu kez `yeniDugum`'dan `bas_`'a taşır — yeni düğüm artık listenin resmî başıdır, kendi altında da eski zincirin tamamını taşımaktadır.

Bu sırayı tersine çevirseydiniz ne olacağını düşünün: önce `bas_` atamasını yapsaydınız, `bas_` yeni düğümü alırdı, ama bu satır *aynı zamanda* eski `bas_`'ın tuttuğu

zinciri (henüz yeniDugum->sonraki'ye taşınmamışken) sahipsiz bırakırdı — eski zincire artık hiçbir işaretçi erişemezdi, düğümleri sızmış (bir daha asla delete edilemeyecek, ama artık erişilemez de olan) bir hâlde kalırdı. Kodda gördüğünüz sıra — önce eski başı yeni düğümün altına taşı, sonra yeni düğümü başa taşı — zincirin hiçbir anda kopmamasını garanti eder.

### 4.4.4 yazdir: sahiplik almadan gezinmek

Listeyi ekrana basmak için zinciri baştan sona *dolaşmanız* gerekir — ama dolaşmak, düğümlerin sahipliğini elinize almanızı gerektirmez, yalnızca onları sırayla *okumanızı* gerektirir. Bu ayrımı somutlaştıran gövdeye bakın:

```
17 void IntBagliListe::yazdir() const {
18    const Dugum* gozlemci = bas_.get(); // sahiplik almayan gozlemci isaretci
19    while (gozlemci != nullptr) {
20       std::cout << gozlemci->deger << " -> ";
21       gozlemci = gozlemci->sonraki.get();
22    }
23    std::cout << "nullptr\n";
24 }
```

gozlemci değişkeninin tipine dikkat edin: bir unique_ptr değil, ham bir const Dugum*'tır — bas_.get() çağrısıyla elde edilir. .get(), bir unique_ptr'ın *sahipliğini devretmeden*, yalnızca tuttuğu ham adresi ödünç size verir; bu adres unique_ptr'ın kendisi değildir, unique_ptr'ın *gösterdiği* yerin bir kopyasıdır. Buna **gözlemci işaretçi** denir. Döngü gozlemci'yi gozlemci->sonraki.get() ile bir sonraki düğüme kaydırarak ilerler — her adımda yine yalnızca bir adres okunur, hiçbir sahiplik el değiştirmez — ve gozlemci nullptr olduğunda (zincirin sonuna gelindiğinde) döngü durur.

**gözlemci işaretçi**: sahiplik almadan yalnızca bir kaynağa erişmek/gezinmek için kullanılan ham işaretçi (*observer / non-owning pointer*).

**Analoji**

Gözlemci ham işaretçiyi bir kitapçıda rafları gezen bir müşteriye benzetebilirsiniz. Müşteri kitapları görebilir, elden ele göz atabilir, hatta okuyabilir — ama hiçbirinin sahibi değildir; dükkândan çıkarken elinde hiçbir şey götürmez, kitapları yerlerinden almaya ya da onlar için sorumluluk taşımaya hiç gerek duymaz. yazdir()'deki gozlemci tam olarak bu müşteridir: zinciri baştan sona gezer, her düğümü okur, ama hiçbir düğümü "kendine almaz". Bu ayrım önemlidir çünkü zincirin gerçek sahipleri — unique_ptr üyeleri — yalnızca bu gezinme işlemi *bittikten sonra* da zinciri sağlam tutmaya devam etmelidir; gezinen taraf o sorumluluğu hiç üstlenmemelidir.

Neden burada bir unique_ptr değil de ham bir gözlemci işaretçi kullanıldığını bir adım daha açın: eğer yazdir() zinciri bir unique_ptr ile gezmeye çalışsaydı, her adımda ya düğümü *kopyalaması* gerekirdi (ama unique_ptr kopyalanamaz, Ünite 3'ten hatırlayın) ya da düğümü *taşıması* gerekirdi (ama taşımak sahipliği gerçekten devreder — zincirin kendisini bozardı, çünkü yazdir() yalnızca const bir üye fonksiyondur, listeyi değiştirmemesi beklenir). Ham gözlemci işaretçi bu ikilemi baştan ortadan kaldırır: sahiplik taşımadığı için ne kopyalama ne taşıma sorunu doğar, yalnızca bir adresi okur ve bir sonrakine geçer.

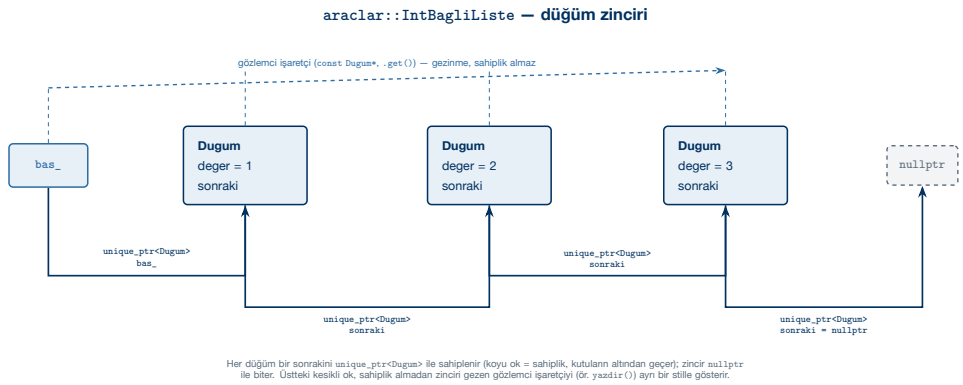
Şimdi listeyi uçtan uca kullanımda görün:

```
6    araclar::IntBagliListe liste;
7    liste.ekleBasa(3);
8    liste.ekleBasa(2);
```

```
9   liste.ekleBasa(1);
10  liste.yazdir();
```

Bu dört satırı sırasıyla izleyin: `liste.ekleBasa(3);` çalıştığında liste `3 -> nullptr` hâline gelir (tek düğüm, kendisi baş). `liste.ekleBasa(2);` çalıştığında yeni bir `2` düğümü kurulur, eski baş (`3`) onun `sonraki`'sine kayar, yeni düğüm başa geçer — liste artık `2 -> 3 -> nullptr`'dır. `liste.ekleBasa(1);` ile aynı adım tekrarlanır — liste `1 -> 2 -> 3 -> nullptr`'a döner. Dikkat edin: `ekleBasa` her zaman *başa* eklediği için, listenin son hâli eklenen değerlerin *tersi* sırasındadır — son eklenen (`1`) en başta durur. `liste.yazdir();` çağrısı bu zinciri gözlemci işaretçiyle gezip `1 -> 2 -> 3 -> nullptr` satırını ekrana basar.

Şekil 4.3 bu zinciri görsel olarak özetliyor: `bas_`'tan başlayarak her düğümden sonrakine uzanan sahiplik oklarını (`unique_ptr<Dugum> sonraki`) ve zincirin üstünden geçen, sahiplik taşımayan ayrı bir gözlemci gezinme okunu.




Şekil 4.3: `IntBagliListe` düğüm zinciri: `bas_`'tan başlayan `unique_ptr` sahiplik okları ve son düğümden `nullptr`'a uzanan uç; ayrı stildeki kesikli ok, sahiplik taşımayan gözlemci gezinmeyi gösterir.

**İpucu**

Bu haftaki `IntBagliListe` yalnızca `int` tutuyor, ama gördüğünüz düğüm + `unique_ptr` deseni tipten bağımsızdır. Aynı desen, ilerde `int deger` yerine bir `Kitap` (ya da bir `Kitap`'a işaret eden bir `shared_ptr<Kitap>`) tutacak biçimde uyarlanabilir — kütüphane temasının bağlı-liste evrimi tam da buradan başlar. Bu resmî olarak şablonlar sonrası, Hafta 9'da `Liste<T>` olarak ele alınacaktır; bu hafta yalnızca desenin somut, çalışan bir örneğini görmeniz yeterlidir.

Bu ünitede gördüğünüz fikir kısaca şudur: `IntBagliListe`, kütüphanenin ilk bağlı veri yapısı olarak, her düğümü bir öncekinin `unique_ptr` ile sahiplendiği bir zincir kurar; `ekleBasa` bu zinciri — eski başı yeni düğümün altına taşıyıp sonra yeni düğümü başa taşıyarak — hiç koparmadan kaydırır, `yazdir` ise sahiplik almayan bir gözlemci ham işaretçiyle zinciri yalnızca okuyarak gezer. Yıkıcıyı hiç yazmamanız, Ünite 3'te gördüğünüz Rule of 0'ın burada da geçerli olduğunu gösterir — tek bir `unique_ptr` üyesinin (`bas_`) zincirleme yıkımı, tüm listeyi otomatik olarak temizler.

**3 • Katılım — Şimdi Siz Deneyin**

`IntBagliListe`'ye 3, sonra 2, sonra 1 değerlerini `ekleBasa` ile eklediniz. (a) `yazdir()` çıktısı ne olur? (b) `yazdir()`, zinciri gezerken neden `bas_` yerine `.get()` ile alınan ham bir `Dugum*` kullanır?

**Geribildirim.** (a) `1 -> 2 -> 3 -> nullptr`: `ekleBasa` her zaman başa eklediği için sıra, ekleme sırasının *tersidir* (son eklenen 1 en başta). (b) Gezinme sahiplik gerektirmez; yalnız okumak/ilerlemek istersiniz. `unique_ptr`'ı kopyalayamazsınız, bir yerel `unique_ptr`'a atarsanız sahipliği zincirden çalıp yapıyı bozarsınız. `.get()` ile alınan, sahiplenmeyen bir `Dugum*` (gözlemci işaretçi) düğümleri "sahiplenmeden" gezer — sahiplik zinciri bozulmadan kalır.

### Kendinizi Sınayın 4.4

`IntBagliListe::yazdir()` neden `bas_` yerine ham bir gözlemci işaretçiyle (`Dugum*`, `.get()` ile alınan) gezinir? Yanıtınızda gezinmenin neden sahiplik gerektirmediğini ve `unique_ptr`'ın kopyalanamaz/yalnızca taşınabilir doğasının doğrudan kullanılsaydı zincirin sahiplik yapısını neden bozabileceğini ele alın.

### 5 • İzleme Aktarım — Başka Nerede?

Aynı "düğüm + `unique_ptr<Dugum> sonraki`" deseni yalnız `int` için değildir; `int deger` yerine bir `Kitap` (ya da paylaşım gerekiyorsa `shared_ptr<Kitap>`) tutacak biçimde uyarlanabilir — Kitaplık temasının bağlı-liste evrimi buradan başlar (resmî generic sürüm Hafta 9'da `Liste<T>` ile gelir). Gözlemci ham işaretçiyle gezinme alışkanlığı da ileride her ağaç/graf gibi bağlı yapıda karşınıza çıkacaktır: sahiplik ve gezinme her zaman ayrı sorumluluklardır.

## Bölüm Özeti

### Özet

Geçen hafta `Kitap` sınıfıyla kurucu ve yıkıcının biçimsel kurallarını gördünüz; bu hafta o temelin üzerine kopyalama ve kaynak yönetimi kavramlarını inşa ettiniz. İlk ünitede Hafta 2'den tanıdığınız `araclar::IntTampon`'u yeniden ele aldınız: derleyicinin sizin hiçbir şey yazmadan ürettiği varsayılan kopya kurucu ve kopya atama operatörünün, bir işaretçi üyesi için yalnızca adresi kopyaladığını — sığ kopya — ve bunun paylaşılan duruma ve çifte `delete`'e nasıl yol açabileceğini gördünüz; çözüm olarak kendi kopya kurucunuzu ve kopya atama operatörünüzü, ikisinde de derin kopya yaparak, elle yazdınız — atama operatöründe ayrıca öz-atama kontrolü ve eski kaynağı bırakma adımı gerekti. İkinci ünite bir adım geri çekilip bu üç fonksiyonun — yıkıcı, kopya kurucu, kopya atama — neden hep birlikte düşünülmesi gerektiğini (Rule of 3) açıkladı; `=default` varsayılan davranışın zaten doğru olduğu sınıflarda bu tercihi açıkça belirtmenizi, `=delete` ise bir işlemi (ör. kopyalamayı) tamamen yasaklamanızı sağladı; Rule of 0 ve Rule of 5 isim düzeyinde tanıtıldı, taşıma (move) semantiğine ise yalnızca kısa/opsiyonel bir kutu düzeyinde değinildi. Üçüncü ünitede bu sorumluluğu kütüphaneye devrettiniz: `unique_ptr` tekil, kopyalanamaz ama taşınabilir bir sahiplik sundu ve `AkilliTampon` örneğinde Rule of 0'ın kendiliğinden nasıl gerçekleştiğini gösterdi; `shared_ptr` ise referans sayımıyla paylaşılan bir sahiplik sundu — `IntTampon b = a;`'nın nesne düzeyinde derin kopyası ile `shared_ptr` kopyasının aynı kaynağı paylaşması arasındaki farkı gördünüz. Dördüncü ünite bu araçları kütüphanenin ilk bağlı ve-

ri yapısında, IntBagliListe'de, bir araya getirdi: her düğümü bir öncekinin unique_ptr ile sahiplendiği bir zincir kurdunuz, ekleBasa zinciri koparmadan kaydırdı, yazdir ise sahiplik almayan bir gözlemci ham işaretçiyle zinciri yalnızca okuyarak gezdi — yıkıcıyı hiç yazmamanız, Rule of 0'ın burada da geçerli olduğunu gösterdi. Bu hafta kaynaklarınızı güvenle kopyaladınız ve akıllı işaretçilerle sardınız; gelecek hafta bu nesnelerin [], <<, karşılaştırma gibi dilin yerleşik operatörleriyle doğal biçimde kullanılmasını (Operatör Yükleme) göreceksiniz.

## Kendinizi Sınayın Cevapları

**Soru (Ünite 1 — varsayılan kopya kurucu neden tehlikelidir).** Bir sınıf ham bir işaretçi ya da kaynak tutuyorsa, kullanıcı kendi kopya kurucusunu/atamasını tanımlamadığı sürece derleyici bunları kendisi üretir ve bu üretilmiş sürümler *üye üye* (memberwise) çalışır — bir işaretçi üyesi için bu, işaretçinin *kendisini* (tuttuğu adresi) kopyalamak demektir, işaret ettiği veriyi değil; buna sığ kopya denir. IntTampon örneğinde bu, iki nesnenin (a ve b) aynı veri adresini paylaşması anlamına gelir: biri a.veri[0]'ı değiştirirse bu değişiklik b.veri[0]'da da görünür (istenmeyen paylaşılan durum), ve ikisi de kapsam dışına çıktığında ikisinin de yıkıcısı aynı adresi delete[] etmeye çalışır — aynı bellek bloğu iki kez serbest bırakılır (çifte delete, tanımsız davranış). Çözüm, kopya kurucu ve kopya atama operatörünü elle yazıp ikisinde de yeni bir new[] ile derin kopya yapmaktır.

**Soru (Ünite 2 — Rule of 3 ve TekilKart'ta =delete).** Rule of 3, bir sınıfın yıkıcısını, kopya kurucusunu veya kopya atama operatöründen *birini* elle yazmak zorunda kaldıysanız genellikle *üçünü de* elle yazmanız gerektiğini söyler — çünkü üçü de aynı soruyu, "bu sınıf bir kaynağın sahibi mi ve bu kaynağa kimin nasıl dokunacağını nasıl garanti ederim" sorusunu yanıtlar; biri elle yazılıp diğerleri derleyicinin varsayılanına bırakılırsa tutarsızlık (sızıntı ya da çifte delete) riski doğar. TekilKart örneğinde kopya kurucu ve kopya atama operatörünü =delete ile tamamen yasaklamanızın nedeni, bu sınıfın — bir izin kartı ya da bilet gibi — kopyalanması *anlamsız*, hatta tehlikeli olan tekil bir kaynağı temsil etmesidir; =delete, bu işlemi çalışma zamanına bırakmak yerine derleme aşamasında, daha kod yazılırken engeller — kimse yanlışlıkla ikinci bir "kopya" kart oluşturamaz.

**Soru (Ünite 3 — unique_ptr vs shared_ptr).** Temel fark sahiplik modelindedir: unique_ptr bir kaynağı *tekil* biçimde sahiplenir — kopyalanamaz (kopya kurucu/atama örtük =delete), yalnızca std::move ile *taşınabilir* (sahiplik devri); shared_ptr ise bir kaynağı dahili bir referans sayacıyla *birden çok* sahip arasında paylaştırır, sayaç sıfıra düştüğünde (son sahip de kapsam dışına çıktığında) kaynak otomatik bırakılır. Hangisini seçeceğinize karar verirken varsayılan tercihiniz her zaman unique_ptr olmalıdır — en ucuz (sayaç tutmaz), en açık (tek sahip, kod okuyanın kim sahiplendiğini net görebildiği) seçenektir; shared_ptr'ı yalnızca gerçekten birden çok yerin *aynı anda* aynı kaynağı yaşatması gerektiği durumlarda tercih edersiniz.

**Soru (Ünite 4 — yazdir() neden gözlemci işaretçiyle gezinir).** Bu üye fonksiyon, listenin başı bas_ üzerinden değil ham bir gözlemci işaretçiyle (const Dugum*, .get() ile alınan) gezinir, çünkü zinciri *gezmek/okumak* sahiplik gerektirmez —

yalnızca her düğümün adresini sırayla okumanız yeterlidir. Eğer gezinme bir `unique_ptr` ile yapılmaya çalışılsaydı iki seçenek kalırdı: düğümü kopyalamak (ama `unique_ptr` kopyalanamaz) ya da taşımak (ama taşımak sahipliği gerçekten devreder, zincirin sahiplik yapısını bozar — oysa `yazdir() const` bir fonksiyondur ve listeyi değiştirmemesi beklenir). Ham gözlemci işaretçi hiçbir sahiplik taşımadığı için bu ikilemi baştan ortadan kaldırır: zincirin gerçek sahipleri — her düğümün `sonraki` üyesindeki `unique_ptr`'lar — gezinme işlemi boyunca hiç dokunulmadan sağlam kalır.